

JAVA DSA

LOOPS & PATTERN PRINTING

Student Class Notes • Learn → Dry Run → Code → Challenge

What you will learn

- Why loops are needed
- for, while and do-while
- break and continue
- Nested loops
- Pattern printing logic
- Number and star patterns
- How to dry-run loop code

Class Rule: Don't memorize patterns. Understand the logic behind rows, columns, spaces and values.

1. What is a Loop?

A loop is used when we want to execute the same block of code multiple times. Instead of writing the same statement again and again, we ask Java to repeat it.

Without a loop

```
System.out.println("Java");
System.out.println("Java");
System.out.println("Java");
System.out.println("Java");
System.out.println("Java");
```

With a loop

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Java");
}
```

Remember: A loop usually has an initialization, a condition, a body, and an update.

2. for Loop

Use a **for loop** when the repetition is naturally count-based.

Syntax

```
for (initialization; condition; update) {
    // repeated statements
}
```

Example: 1 to 10

```
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```

Dry Run: For each iteration ask: What is i? Is the condition true? What prints? What happens after i++?

3. while Loop

A while loop checks the condition before every iteration.

Example

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

■■ If the condition never becomes false, the loop can become an infinite loop.

4. do-while Loop

A do-while loop executes its body first and checks the condition afterward. Therefore, the body runs at least once.

Example

```
int i = 1;

do {
```

```
        System.out.println(i);
        i++;
    } while (i <= 5);
```

5. Quick Comparison

Loop	Condition	Best for
for	Before	Counting / known repetitions
while	Before	Condition-driven repetition
do-while	After	Must execute at least once

6. break and continue

break → stops the loop completely.

continue → skips the current iteration and moves to the next iteration.

break

```
for (int i = 1; i <= 10; i++) {
    if (i == 6) {
        break;
    }
    System.out.println(i);
}
```

continue

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue;
    }
    System.out.println(i);
}
```

7. Loop Practice — Try Yourself

1. Print numbers from 1 to 20.
2. Print numbers from 20 to 1.
3. Print all even numbers from 1 to 50.
4. Print all odd numbers from 1 to 50.
5. Print the multiplication table of a number.
6. Find the sum of 1 to n.
7. Find factorial of n.
8. Count numbers divisible by 3 from 1 to n.

8. Nested Loops

A nested loop means one loop is inside another. For pattern printing, think: **outer loop = rows** and **inner loop = items in each row**.

3 rows × 4 columns

```
for (int row = 1; row <= 3; row++) {
    for (int col = 1; col <= 4; col++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Output:

```
* * * *
* * * *
* * * *
```

9. Pattern Printing — The 4-Step Method

Step 1: Count the rows.

Step 2: For each row, count what must be printed.

Step 3: Decide what the inner loop prints.

Step 4: Print a new line after each row.

Example: If row 1 has 1 star, row 2 has 2 stars, row 3 has 3 stars... then the inner loop usually runs up to row.

10. Pattern 1 — Square / Rectangle

5 rows × 5 columns

```
for (int row = 1; row <= 5; row++) {
    for (int col = 1; col <= 5; col++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Output:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

11. Pattern 2 — Increasing Triangle

1, 2, 3, 4, 5 stars

```
for (int row = 1; row <= 5; row++) {
    for (int col = 1; col <= row; col++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Key idea: **number of stars = row number**.

12. Pattern 3 — Decreasing Triangle

5, 4, 3, 2, 1 stars

```
for (int row = 5; row >= 1; row--) {
    for (int col = 1; col <= row; col++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

13. Pattern 4 — Number Triangle

1 / 1 2 / 1 2 3 ...

```
for (int row = 1; row <= 5; row++) {
    for (int col = 1; col <= row; col++) {
        System.out.print(col + " ");
    }
    System.out.println();
}
```

Here the printed value is **col**, not row.

14. Pattern 5 — Same Number in Each Row

1 / 2 2 / 3 3 3 ...

```
for (int row = 1; row <= 5; row++) {
    for (int col = 1; col <= row; col++) {
        System.out.print(row + " ");
    }
}
```

```
    }  
    System.out.println();  
}
```

Here the printed value is **row**.

15. Pattern 6 — Right-Aligned Triangle

For alignment, use a separate loop for spaces before the stars.

Spaces + stars

```
int n = 5;

for (int row = 1; row <= n; row++) {

    for (int space = 1; space <= n - row; space++) {
        System.out.print(" ");
    }

    for (int star = 1; star <= row; star++) {
        System.out.print("* ");
    }

    System.out.println();
}
```

16. Pattern 7 — Pyramid

A pyramid usually needs spaces plus an odd number of stars: 1, 3, 5, 7, ...

Pyramid

```
int n = 5;

for (int row = 1; row <= n; row++) {

    for (int space = 1; space <= n - row; space++) {
        System.out.print(" ");
    }

    for (int star = 1; star <= 2 * row - 1; star++) {
        System.out.print("*");
    }

    System.out.println();
}
```

17. Pattern 8 — Continuous Number Grid

1 2 3 / 4 5 6 / ...

```
int n = 3;
int value = 1;

for (int row = 1; row <= n; row++) {

    for (int col = 1; col <= n; col++) {
        System.out.print(value + " ");
        value++;
    }

    System.out.println();
}
```

18. Pattern Debugging

- Wrong number of lines? → Check the outer loop.
- Wrong number of items in a line? → Check the inner loop.

- Everything is on one line? → Check `println()`.
- Pattern is shifted? → Check spaces.
- Wrong values? → Decide whether you need row, col, or a separate counter.
- One extra or one missing item? → Check `<` versus `<=`.

19. Challenge Zone ■

Try these without looking at the examples above. First draw the expected output, then write the loops.

1. Print a 4×6 rectangle of stars.
2. Print an increasing triangle of 1 to 7 stars.
3. Print a decreasing triangle of 7 to 1 stars.
4. Print: 1 / 2 2 / 3 3 3 / 4 4 4 4.
5. Print: 1 / 1 2 / 1 2 3 / 1 2 3 4.
6. Print a right-aligned triangle.
7. Print a pyramid of 5 rows.
8. Print a 5×5 multiplication table.
9. Print numbers from 1 to 50, but skip multiples of 5.
10. Print numbers from 1 to 100 and stop when you reach 73.

20. Mini Project — Number Analyzer

Create a program that takes a number **n** and uses loops to:

- Print 1 to n
- Print even numbers to n
- Find the sum from 1 to n
- Count how many numbers are divisible by 3
- Print a multiplication table

Bonus: Add a menu using switch-case and use break to exit the menu.

21. What to Remember

- **for**: best for count-based repetition.
- **while**: condition checked before the body.
- **do-while**: body executes at least once.
- **break**: exit the loop.
- **continue**: skip the current iteration.
- **Nested loops**: outer = rows, inner = columns/items.
- **Patterns**: identify rows first, then the contents of each row.

22. Exit Questions

1. Why do we need loops?
2. When would you choose a for loop?
3. Why can a while loop become infinite?
4. Why does do-while execute at least once?
5. What is the difference between break and continue?

6. In pattern printing, what usually controls the rows?
7. How do you decide the inner-loop limit for an increasing triangle?